

ACE Agent (Automated Clustering Expert)

生产级智能体聚类系统工程手册 v2.0

系统架构组 - Gemini CLI 实验室

2026 年 4 月 17 日

Abstract

本文档是 ACE Agent 的核心技术指南。系统采用 Orchestrator-Worker 架构，通过语义路由分发任务，并依靠专家智能体的自愈循环 (Self-healing Loop) 在隔离沙箱内生成并执行数据科学代码。本版本 (v2.0) 特别强化了对 RAG 知识检索、意图动态判定以及专家算法职责域的实现细节描述。

Contents

1	引言与背景	2
2	系统整体架构 (Architecture Deep Dive)	2
2.1	Orchestrator-Worker 模式	2
2.2	数据流拓扑图	2
3	核心功能模块详解	2
3.1	语义路由与意图判定 (MasterRouter)	2
3.2	专家自愈循环 (Self-Correction Loop)	2
3.3	安全沙箱执行 (CoderSandbox)	2
4	LLM 接入点与 Prompt 策略	3
4.1	MasterRouter 提示词工程	3
4.2	专家代码生成提示词	3
5	函数调用时序与协作	3
5.1	初始化阶段	3
6	关键类与方法索引 (API Reference)	3
7	复现与环境配置指南	3
7.1	环境配置	3
7.2	中文字体支持	3

1 核心架构与逻辑平面

1.1 语义路由 (MasterRouter) 的判定逻辑

MasterRouter 是系统的指挥中心，位于 `agent_core/router.py`。它不依赖传统正则表达式，而是通过 LLM 语义分析将输入映射为 JSON 格式的意图对象。

意图分类逻辑

- **NEW_TASK**: 当用户输入包含“分析”、“运行”、“聚类”或具体算法名（如“谱聚类”）时触发。
- **FOLLOW_UP**: 当输入包含“解释”、“为什么”、“可视化”或针对已有结果的追问时触发。

函数实现细节: `analyze_intent` 系统将最近 3 轮对话历史 (`history`) 压缩后连同当前 Prompt 送入 LLM，并使用正则 `re.search(r'{.*}', ...)` 强制提取 JSON 块。

1.2 主控编排器 (ACESupervisor) 的调度算法

ACESupervisor 负责协调知识引擎、路由与专家代理。

核心调度流 (run 方法)

1. **意图捕获**: 调用 MasterRouter。
2. **知识注入 (RAG)**: 调用 `KnowledgeEngine.query`，若匹配到学术背景，则将知识文本前置于用户 Prompt。
3. **专家激活**:
 - 如果是 FOLLOW_UP，直接进入 LLM 回答逻辑。
 - 如果是 NEW_TASK，并发激活 `CentroidExpert` 和 `TopologyExpert`。
4. **评分与摘要**: 对所有算法结果按 `metrics['score']` 降序排列，取首位作为优胜算法生成摘要。

2 专家智能体实现细节 (Worker Implementation)

2.1 专家职责域划分

系统将算法按数学逻辑划分为两个主要专家：

- **质心专家 (CentroidExpert)**: 负责 KMeans, GMM, MiniBatchKMeans。
- **拓扑专家 (TopologyExpert)**: 负责 DBSCAN, SpectralClustering (谱聚类), Agglomerative, OPTICS。

2.2 自愈循环 (Self-Correction Loop) 实现

实现在 `expert_sub_agents/base.py` 的 `execute_with_self_correction` 方法中。

算法流程

```
1 for attempt in range(1, 4):
2     code = generate_or_fix_code(prompt, error_msg)
3     run_result = sandbox.execute(code, X, y)
4     if run_result.success:
5         save_artifacts()
6         return results
7     else:
8         extract_traceback() # 提取最后 200 字符报错
```

3 RAG 知识引擎 (Knowledge Engine)

3.1 知识扫描与去重

KnowledgeEngine 使用 `ingested_manifest.json` 维护一个 MD5 哈希列表。扫描 `docs/` 目录时，仅处理哈希值变动或新增的文件。

3.2 向量检索配置

- **Embedding 模型:** `all-MiniLM-L6-v2` (本地运行)。
- **持久化库:** ChromaDB。
- **环境变量注入:** 强制设置 `HF_HUB_OFFLINE=1` 以确保离线环境下的启动速度。

4 安全沙箱与代码隔离 (CoderSandbox)

4.1 执行环境构造

在 `tools/coder_sandbox.py` 中，通过 `exec()` 运行代码前会初始化一个受限字典 `exec_env`。

注入变量清单

- `X, y`: 内存中的 `numpy` 数组。
- `plt, np`: 预导入的数据科学库。
- `artifacts`: 空字典，用于专家回传结果。

输出捕获机制 专家生成的代码必须将结果存入 `artifacts['算法名']`，包含 `labels`、`metrics` 和 `plot_path` 三个必填字段，否则会被判定为执行失败。

5 API 参考手册 (API Reference)

5.1 UniversalLLMClient

- `chat_completion(messages, system_prompt)`: 具备自动重试机制的通用接口。
- `summarize_report(payload)`: 接收 `user_intent` 参数，实现意图优先的报告摘要。

5.2 LatexReportGenerator

- `generate(report)`: 动态扫描 `report.results`，为每个算法生成 `\subsection`，并将 `plot_path` 转换为 `LATEX` 路径。

6 部署与排烟测试

6.1 中文字体支持方案

由于在 Windows 环境下 `matplotlib` 默认不识别中文，系统在启动时会检测 Win32 平台并尝试加载“微软雅黑”或“黑体”。

6.2 离线模式 (Offline Mode)

通过屏蔽 `transformers` 的日志输出 (`VERBOSITY=error`)，解决了大规模知识库加载时的终端卡顿问题。